

Bioinformatics session

4th annual workshop on
bioinformatics and variant
interpretation in InPreD

https://inpred.github.io/26-09_bioinfo_ws/



2. Containerization

A brief history

1979	<code>chroot</code> system call (changing the root directory of a process and its children to a new location in the filesystem) in Uni V7 considered as beginning of process isolation
2000	FreeBSD Jails allows administrators to partition FreeBSD computer system into several independent, smaller systems (<i>jails</i>) – with the ability to assign IP address for each system and configuration
2001	Linux VServer is jail mechanism that can partition resources, similar to FreeBSD Jails

A brief history

2004	First public beta of Solaris Containers released - combines system resource controls and boundary separation provided by zones
2005	Open Virtuozzo is operating system-level virtualization technology for Linux which uses patched Linux kernel for virtualization, isolation, resource management and checkpointing
2006	Process Containers, launched by Google, was designed for limiting, accounting and isolating resource usage of collection of processes - renamed to <i>Control Groups (cgroups)</i> and merged into Linux kernel

A brief history 📖

2008	LinuX Containers (LXC) was first, most complete implementation of Linux container manager - implemented using cgroups and Linux namespaces
2011	CloudFoundry started Warden can isolate environments on any operating system running as daemon and providing API for container management
2013	Let Me Contain That For You (LMCTFY) kicked off as open-source version of Google's container stack - providing Linux application containers

A brief history 📅

2013	Docker emerged
2016	Singularity (later Apptainer) was released

Docker

What is it?

- containerization technology to package application and dependencies into a container
- the container image can be shipped and run consistently across different computing environments

Which benefits does it provide?



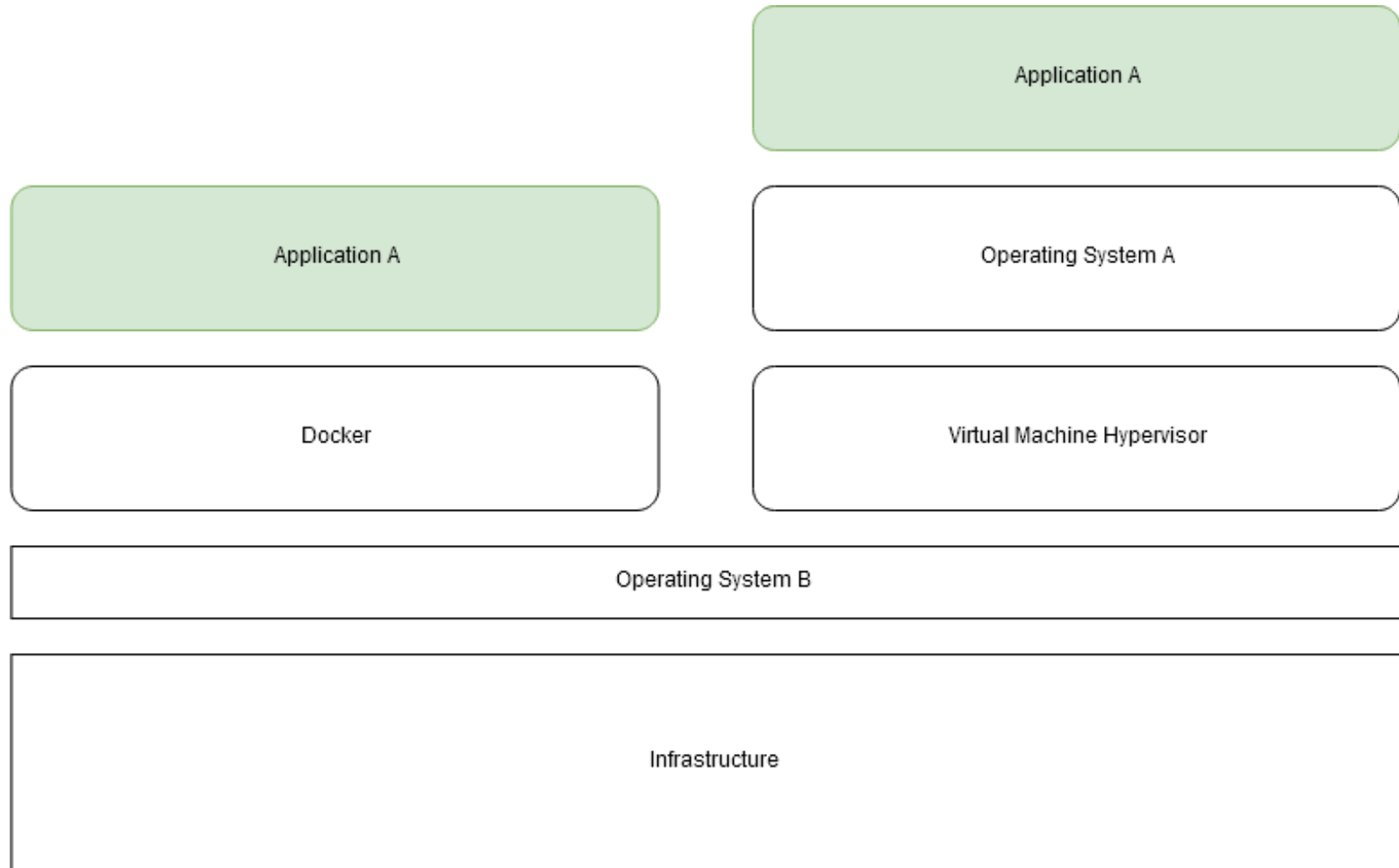
1. Consistency
2. Isolated environments
3. Portability
4. Efficiency



How is it different from virtual machines?

- containers use and share host OS's kernel making them more lightweight and efficient
- virtual machines emulate entire physical machine, including OS making it possible to run multiple OS instances on single physical machine

How is it different from virtual machines?



Key Concepts 🔑

1. **Image:** standardized package that includes all files, binaries, libraries, and configurations to run container
2. **Container:** running instance of image providing isolated runtime environment
3. **Dockerfile:** instructions on how to build image
4. **Docker Hub:** public registry where developers can share and access pre-built images

Key Components 🔑

1. **Docker Engine:** core, open-source technology that builds and runs containers
2. **Docker Client:** command-line tool that sends instructions to the Docker Daemon using REST APIs
3. **Docker Daemon (`dockerd`):** receives and processes API requests and calls container runtime
4. **Container Runtime (`containerd`):** turns static container image into running, isolated application on host OS; industry standard

Let's explore! 

Start by going to https://github.com/InPreD/26-09_bioinfo_ws_docker_and_ci and create a fork

3. Continuous Integration (CI)

